Attack Graph Analysis

A Graph Theory Research Paper

Mohamed Khaled **Malak Ahmed** **Hoda Hussein** **Ali Elkhouly**

May 2026

Abstract

Cybersecurity attack analysis is one of the practical applications of graph theory, where different graph-theoretic theorems and analysis methods are utilized to understand threat models in a specific network. This project applies graph theory to cybersecurity through Attack Graph Analysis. Network threats are modeled as a directed graph, utilizing vertices to represent vulnerabilities and edges to denote possible exploit transitions between them. Using a shortest-path algorithm, we identify the most efficient routes to critical assets that may be exploited by an external attacker. Furthermore, the project demonstrates how calculating a minimum vertex cut provides a direct mathematical proof for the exact minimum number of security patches required to sever all attack paths. The proof and results are established using Menger's Theorem, which states that the maximum number of internally vertex-disjoint paths between two vertices equals the minimum number of vertices whose removal disconnects them.

Contents

1 Introduction

As network infrastructures become increasingly complex, cyberattacks rarely rely on a single vulnerability. Instead, attackers exploit multiple vulnerabilities to move deeper into a network. These attack strategies have motivated security teams to model vulnerabilities logically, leading to the adoption of graph-theoretic security frameworks.

This paper applies Attack Graph Analysis to network security by modeling a university campus network as a directed weighted graph. In this model:

- Vertices represent known vulnerabilities.
- Directed edges represent exploit paths connecting them.

By assigning weights to edges according to exploit difficulty, we can identify not only possible attack paths, but also the most likely paths an adversary would choose.

Our analysis focuses on two primary areas:

1. Using the Bellman-Ford shortest-path algorithm to determine the path of least resistance for an attacker attempting to reach a protected database from a public-facing web server.
2. Applying Menger's Theorem to compute a minimum vertex cut, identifying the smallest set of vulnerabilities that must be patched to completely disconnect the attacker from the target.

This transforms the practical security question: "What should we patch first?" into a solvable combinatorial optimization problem. Ultimately, this demonstrates how graph theory can serve as a rigorous and practical tool for quantitative cybersecurity analysis. It gives security teams mathematically backed, concrete guidance on exactly where to focus their defensive efforts.

2 Definitions

2.1 Attack Graph

An attack graph is a directed graph $G = (V, E)$ where:

- V is a finite non-empty set of vertices, each representing a distinct system state of a known vulnerability.
- $E \subseteq V \times V$ is a set of directed edges where each ordered pair (u, v) such that $u \neq v$ represents a feasible exploit transition from u to v.
- $w : E \to \mathbb{R}^+$ is a weight function assigning a positive real cost to each edge, typically representing exploit difficulty.
- The source vertex $s \in V$ represents the attacker's initial foothold.
- The target vertex $t \in V$ represents the protected critical asset.

An attack path from s to t is a directed walk $P = \langle v_0, v_1, \ldots, v_k \rangle$ such that $v_0 = s$, $v_k = t$, and $(v_i, v_{i+1}) \in E$ for all $0 \leq i < k$. The length of P in the unweighted case is k (number of edges); in the weighted case, it is the sum of edge weights along the path:

$$\sum_{i=0}^{k-1} w(v_i, v_{i+1}) \tag{1}$$

A vertex $v \in V$ is reachable from s if there exists at least one directed path from s to v in G. The reachability set is $R(s) = \{v \in V \mid \exists \text{ a directed path from } s \text{ to } v \text{ in } G\}$. The target t is considered compromisable if and only if $t \in R(s)$.

A set $C \subseteq V \setminus \{s, t\}$ is called an $s - t$ **vertex cut** if removing C leaves no directed path from s to t. The minimum vertex cut is the smallest such set, and its cardinality $|C^*|$ gives the exact minimum number of vulnerabilities that must be patched to guarantee t is unreachable from s.

An attack path $P = \langle v_0, v_1, \ldots, v_k \rangle$ is **simple** if all vertices are distinct ($v_i \neq v_j$ for all $i \neq j$). In attack graph analysis, we restrict attention to simple paths, since revisiting a vulnerability state yields no additional attacker capability and inflates path cost without benefit.

In practice, attack graphs are modeled as **directed acyclic graphs (DAGs)**. Cycles would imply an attacker can revisit a previously exploited state, which adds no new capability. Acyclicity guarantees that shortest-path algorithms terminate correctly without negative-weight cycle concerns.

3 University Campus Network Attack Graph

3.1 Vertices

The vertex set models the network topology infrastructure hosts divided into functional security layers:

$$V = \{s, v_1, v_2, v_3, v_4, v_5, v_6, v_7, t\} \tag{2}$$

- **Source (s):** Public web server (Layer 0), where the attacker starts the exploit process.
- **Target (t):** Records database (Layer 4), the final protected destination.

3.2 Edges

The set of feasible directed vulnerability transitions is defined as:

$$E = \{(s, v_1), (s, v_2), (s, v_3), (v_1, v_4), (v_2, v_4), (v_3, v_5), (v_4, v_5), (v_4, v_7), (v_5, v_6), (v_6, t), (v_7, t)\} \tag{3}$$

3.3 Weights

The specific asymmetric transitional cost difficulty functions are defined as:

$$\begin{aligned} w(s, v_1) &= 2 & w(s, v_2) &= 4 & w(s, v_3) &= 3 \\ w(v_1, v_4) &= 3 & w(v_2, v_4) &= 2 & w(v_3, v_5) &= 5 \\ w(v_4, v_5) &= 4 & w(v_4, v_7) &= 3 & w(v_5, v_6) &= 2 \\ w(v_6, t) &= 1 & w(v_7, t) &= 3 \end{aligned}$$

3.4 Visual Representation

The network relationships, paths, and transitional edge costs defined above are graphically detailed below:

4 Shortest Path Analysis

4.1 Formal Problem Definition

For a given directed weighted graph $G = (V, E, w)$ representing the secured network, with $w : E \to \mathbb{R}^+$, the shortest-path problem asks for a directed path $P^* = \langle v_0, v_1, \ldots, v_k \rangle$ from

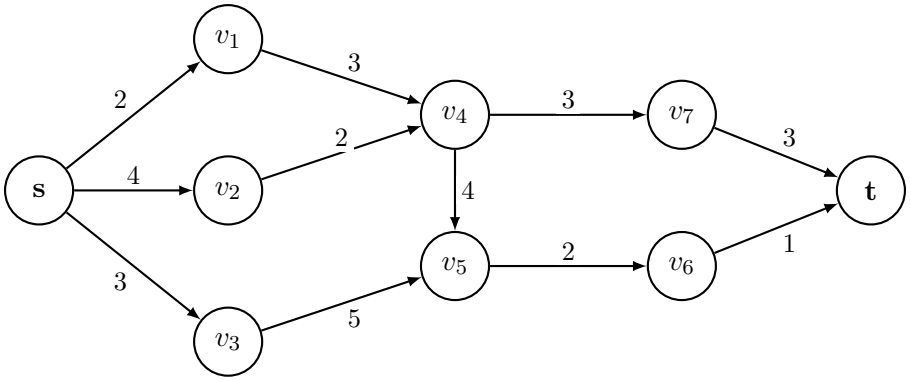

Figure 1: University Campus Network Attack Graph Model

the entrance vertex $v_0 = s$ to the exit target vertex $v_k = t$ such that the total path weight is minimized:

$$\sum_{i=0}^{k-1} w(v_i, v_{i+1}) \quad \text{is minimized.} \tag{4}$$

4.2 Assumptions

1. The target destination, denoted by vertex t, is reachable from the entrance s, meaning there exists at least one directed path from s to t.

4.3 Bellman-Ford Algorithm Theorem

Let $G = (V, E, w)$ be a directed weighted graph with no negative-weight cycles, and let $s \in V$. After the $(|V| - 1)^{\text{th}}$ iteration of the outer loop in the algorithm, for every vertex $v \in V$:

$$dist[v] = d(s, v) \tag{5}$$

where $d(s, v)$ denotes the true shortest-path distance from s to v, and $d(s, v) = \infty$ if v is unreachable from s.

4.3.1 Edge Relaxation Definition

For any vertex $v \in V$, $dist[v]$ denotes the current best estimate for the minimal distance from the starting vertex s to v. Edge relaxation means that for a directed edge (u, v) with weight $w(u, v)$, if $dist[u] + w(u, v) < dist[v]$, then we update $dist[v] \leftarrow dist[u] + w(u, v)$. This step updates distances systematically by checking for a shorter structural shortcut until no further modifications are possible.

4.4 Bellman-Ford Pseudocode

Input: Directed weighted graph $G = (V, E, w)$, source vertex $s \in V$
Output: $dist[v] = d(s, v)$ for all $v \in V$, *predecessor* array
1: **INITIALIZE**:
2: **for each** vertex $v \in V$ **do**
3: $\quad dist[v] \leftarrow \infty$
4: $\quad predecessor[v] \leftarrow$ NIL
5: $dist[s] \leftarrow 0$
6: **RELAX**:
7: **for** $i = 1$ **to** $|V| - 1$ **do**

8: **for each** edge $(u, v) \in E$ **do**
9: **if** $dist[u] + w(u, v) < dist[v]$ **then**
10: $dist[v] \leftarrow dist[u] + w(u, v)$
11: $predecessor[v] \leftarrow u$
12: **RETURN** $dist[]$, $predecessor[]$

4.5 Mathematical Proof by Induction

We use induction on the loop iteration parameter i. We claim that after the i^{th} iteration of the algorithm's loop, $dist[v] = d(s, v)$ for every vertex v whose shortest path from s uses at most i edges.

Base Case ($i = 0$): Before iterating, $dist[s] = 0$, and for any other vertex v, $dist[v] = \infty$. This matches the true shortest distance using 0 edges.

Induction Hypothesis ($i = k$): Assume that after k iterations, for any vertex u whose true shortest path from s uses at most k edges, $dist[u] = d(s, u)$.

Inductive Step ($i = k + 1$): Let v be any vertex whose shortest path from s uses exactly $k+1$ edges. Let this path be $P = \langle s, v_1, \ldots, u, v \rangle$. The subpath from s to u uses exactly k edges and must be the shortest path to u. By the induction hypothesis, $dist[u] = d(s, u)$ at the end of iteration k. In iteration $k + 1$, all edges are scanned, meaning edge (u, v) is relaxed. Thus:

$$dist[v] \leq dist[u] + w(u, v) = d(s, u) + w(u, v) = d(s, v) \tag{6}$$

Since it always holds that $d(s, v) \leq dist[v]$, the relaxation step forces $dist[v] = d(s, v)$.

Termination: Because an acyclic graph contains no paths longer than $|V| - 1$ edges, the values stabilize completely after $|V| - 1$ iterations.

4.6 Applying the Algorithm to the Attack Graph Model

Since our network has $|V| = 9$ vertices, the algorithm runs for at most $|V| - 1 = 8$ iterations.

Initialization: Set $dist[s] = 0$ and $dist[v] = \infty$ for all $v \in V \setminus \{s\}$, and set $predecessor[v] =$ NIL.

Iteration 1 Trace

Scanning all edges E and applying relaxation updates the following structural guesses:

- Edge $(s, v_1) : dist[s] + w(s, v_1) = 0 + 2 = 2 < \infty \rightarrow dist[v_1] = 2,\ \text{pred}[v_1] = s.$
- Edge $(s, v_2) : dist[s] + w(s, v_2) = 0 + 4 = 4 < \infty \rightarrow dist[v_2] = 4,\ \text{pred}[v_2] = s.$
- Edge $(s, v_3) : dist[s] + w(s, v_3) = 0 + 3 = 3 < \infty \rightarrow dist[v_3] = 3,\ \text{pred}[v_3] = s.$
- Edge $(v_1, v_4) : dist[v_1] + w(v_1, v_4) = 2 + 3 = 5 < \infty \rightarrow dist[v_4] = 5,\ \text{pred}[v_4] = v_1.$
- Edge $(v_2, v_4) : dist[v_2] + w(v_2, v_4) = 4 + 2 = 6 > 5 \rightarrow$ No update.
- Edge $(v_3, v_5) : dist[v_3] + w(v_3, v_5) = 3 + 5 = 8 < \infty \rightarrow dist[v_5] = 8,\ \text{pred}[v_5] = v_3.$
- Edge $(v_4, v_5) : dist[v_4] + w(v_4, v_5) = 5 + 4 = 9 > 8 \rightarrow$ No update.

- Edge $(v_4, v_7) : dist[v_4] + w(v_4, v_7) = 5 + 3 = 8 < \infty \rightarrow dist[v_7] = 8,\ \text{pred}[v_7] = v_4$.
- Edge $(v_5, v_6) : dist[v_5] + w(v_5, v_6) = 8 + 2 = 10 < \infty \rightarrow dist[v_6] = 10,\ \text{pred}[v_6] = v_5$.
- Edge $(v_6, t) : dist[v_6] + w(v_6, t) = 10 + 1 = 11 < \infty \rightarrow dist[t] = 11,\ \text{pred}[t] = v_6$.
- Edge $(v_7, t) : dist[v_7] + w(v_7, t) = 8 + 3 = 11 = dist[t] \rightarrow$ No update.

Iterations 2 through 6 Trace

Entering Iteration 2, the current distance vector is:

$$[dist[s] = 0, dist[v_1] = 2, dist[v_2] = 4, dist[v_3] = 3, dist[v_4] = 5, dist[v_5] = 8, dist[v_6] = 10, dist[v_7] = 8, dist[t] =$$

Re-evaluating every edge in Iteration 2 yields no improvements because all calculated costs are greater than or equal to current records (e.g., edge (v_1, v_4) checks $2+3 = 5$, which is not strictly less than 5). Since no updates occur during Iteration 2, the algorithm has converged. Iterations 3, 4, 5, and 6 similarly execute edge scans without making modifications.

4.7 Shortest Paths in the Attack Graph

The terminal analysis identifies two active symmetric path structures sharing identical absolute metric lengths:

Path A

$$s \rightarrow v_3 \rightarrow v_5 \rightarrow v_6 \rightarrow t \tag{7}$$

Cost: $3 + 5 + 2 + 1 = 11$

Path B

$$s \rightarrow v_1 \rightarrow v_4 \rightarrow v_7 \rightarrow t \tag{8}$$

Cost: $2 + 3 + 3 + 3 = 11$

The baseline shortest distance to compromise the target system is:

$$d(s, t) = 11 \tag{9}$$

5 Menger's Theorem and Security Hardening

5.1 Vertex-Disjoint Paths

Two directed paths from s to t are internally vertex-disjoint if they share no intermediate vertices.

5.2 Menger's Theorem

Let D be a finite digraph and let $s, t \in V(D)$. Then:

$$\max\{\text{number of internally vertex-disjoint } s-t \text{ paths}\} = \min\{|C| \mid C \text{ is an } s-t \text{ vertex cut}\} \tag{10}$$

5.3 Application to the Attack Graph

The maximum number of internally vertex-disjoint paths from s to t is 2. These paths are:

$$P_1 : s \to v_2 \to v_4 \to v_7 \to t$$
$$P_2 : s \to v_3 \to v_5 \to v_6 \to t$$

Therefore, $\max k = 2$.

By Menger's Theorem, the minimum vertex cut also has cardinality 2. One valid minimum vertex cut is:

$$C^* = \{v_4, v_5\} \tag{11}$$

Removing v_4 and v_5 completely disconnects all possible attack paths from s to t.

6 Security Interpretation

The minimum vertex cut $C^* = \{v_4, v_5\}$ corresponds directly to the absolute minimum set of system vulnerabilities that must be patched to guarantee the target system becomes completely unreachable and secure against an external attacker.

Table 1: System Vulnerability Mitigation Matrix

Vertex	**System Role**	**Required Action**
v_4	Internal pivot host	Disable lateral movement and isolate the host.
v_5	Active Directory server	Patch the controller and enforce MFA.

7 References

1. Jha, S., Sheyner, O., & Wing, J. (2002). Two formal analyses of attack graphs.
2. Ammann, P., Wijesekera, D., & Kaushik, S. (2002). Scalable graph-based network vulnerability analysis.
3. Bellman, R. (1958). On a routing problem.
4. Cormen, T. H., et al. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
5. El Joubbeh, M. (2019). *Minimal Separators in Graphs.*
6. Göring, F. (2000). Short proof of Menger's theorem.